An End-to-End Data Engineering Pipeline for AI-Assisted Project Domain Classification Using Multi-Source Data

Alaa Alhowaide
Jordan University of Science and Technology
Department of Computer Information Systems
azalhowaide@just.edu.jo

Ahmad Elaina
Jordan University of Science and Technology
Department of Computer Information Systems
afelaina22@cit.just.edu.jo

Ayoub Malkawi
Jordan University of Science and Technology
Department of Computer Information Systems
abmalkawi22@cit.just.edu.jo

Osama Almallah
Jordan University of Science and Technology
Department of Computer Information Systems
omalmallah22@cit.just.edu.jo

Yaqoub Alothman
Jordan University of Science and Technology
Department of Computer Information Systems
yialothman22@cit.just.edu.jo

Abstract

Automated classification of software and IT projects into well-defined technical domains remains an open problem, hampered primarily by the near-total absence of large-scale, publicly labeled datasets tailored to this task. This paper presents an end-to-end data engineering pipeline that confronts this scarcity directly: rather than waiting for organic data collection, we generate a curated corpus of 486 project records spanning 38 distinct technical domains through structured AI-assisted synthesis, then feed that corpus into a full ETL stack and a suite of classification models. The pipeline systematically ingests heterogeneous source formats—structured JSON, tabular CSV, and raw text—normalizes and enriches each record through domain-aware feature engineering, and applies rule-based keyword taxonomies alongside engineered numerical features to prepare machine-learning-ready representations. We evaluate three classifiers of increasing theoretical sophistication: Logistic Regression, Random Forest, and BERT. The results expose a *complexity-constrained performance paradox*: the ensemble method, despite achieving perfect training-set memorization, attains the highest test accuracy at 67.37%, while BERT falls well below expectations at roughly 35%. This inversion where a simpler model outperforms a state-of-the-art transformer traces directly to dataset scale. Beyond classification performance, we analyze data quality challenges, class imbalance effects, ethical considerations of synthetic training data, and the practical case for human-augmented rather than fully automated deployment. The pipeline, evaluation protocol, and findings together establish a reproducible foundation for project-domain classification research in enterprise environments.

ICICS '26, Irbid, Jordan

ACM ISBN 979-8-4007-2350-6
https://doi.org/xxxxxxx/xxxxxxx.xxxxxxx

CCS Concepts

• **Computing methodologies** → **Machine learning**; *Neural networks*
; • **Information systems** → **Clustering and classification**; *Data management systems.*

Keywords

Synthetic Data Generation, Project Domain Classification, Feature Engineering Pipeline, Multi-class Text Categorization, ETL Data Processing

ACM Reference Format:
Alaa Alhowaide, Ahmad Elaina, Ayoub Malkawi, Osama Almallah, and Yaqoub Alothman. 2026. An End-to-End Data Engineering Pipeline for AI-Assisted Project Domain Classification Using Multi-Source Data. In *Proceedings of ICICS '26*. ACM, New York, NY, USA, 7 pages. https://doi.org/xxxxxxx/xxxxxxx.xxxxxxx

1 Introduction

Categorizing software and IT projects into precise technical domains—such as Web Development, Data Analytics, AI/ML, or Cybersecurity is essential for key decisions like resource allocation, cost estimation, technology stack selection, and portfolio planning. Yet manual classification often falters: project descriptions differ wildly in wording, detail, and format depending on the organization or author, leading to inconsistent and time-intensive judgments. While automation promises relief, it faces a core challenge: the absence of large-scale, publicly available labeled datasets tailored to this task.

We tackle this directly: generating a synthetic dataset through controlled AI prompting, building a full ETL pipeline around it, and benchmarking several classifiers to understand what is and isn't achievable at this scale. The work has four concrete goals: (1) building a large, varied dataset using AI-assisted synthesis; (2) designing feature extraction and transformation steps that hold up against messy, technical project text; (3) training and comparing classification models across a range of complexity; and (4) being honest about the limits of synthetic data when it comes to real deployment.

This work bridges practice and research. For project managers, it offers tools to streamline categorization and boost consistency; for scholars, it provides benchmarks for synthetic data efficacy, domain-specific features, and model generalization.

The paper unfolds as follows. Section 2 surveys related work in text classification, synthetic data, and project automation. Section 3 details data generation and dataset properties. Section 4 outlines the ETL system architecture. Section 5 covers cleaning, transformation, and features. Section 6 describes models and evaluation. Section 7 analyzes results, limitations, and implications. Section 8 wraps up with future directions.

2 Related Work

Text classification is still one of the cornerstones of natural language processing (NLP), driving a wide spectrum of applications from sentiment analysis[1], topic modeling[2] and document categorisation. While standard algorithms such as Naive Bayes, SVMs and logistic regression perform well on polished data[3], deep learning (beginning with CNNs and RNNs; now BERT-like transformers) has dominated modern benchmarks[4].

So project domain classification merges tech doc parsing with project automation, as the close cousin to software categorization but spiced up by intricate jargon, narrative-spec hybrids, and unpredictable depth of description. Commercial tools can be limited to using rules or manual checks [5] which highlights the gap in available ML-driven solutions.

Synthetic Data: Saviour for poor regimes, evolving from simple augmentations (paraphrase, back-translate, noise) to GANs and LLMs conjuring full instances[7, 8]. While LLMs are adept at generating fluently, contextaware output, they struggle with distribution shifts[6] and label errors.

ML in project management[9] sub-focusses on risk, effort and scheduling and takes taxonomies from COBIT/ITIL[10]—sound ground but missed parseable labeled sets for training.

At this juncture, we apply LLM synthesis for projects classification, build tech-specific features, and evaluate architectures in the face of data paucity to reveal deployable classifiers.

3 Data Description and Sources

As there were no publicly available labeled databases for technology project classification, we created a synthetic dataset of 486 records using five different sources.

3.1 Data Collection Methodology

Structured JSONs (100 projects). These were obtained via GPT-3.5 and GPT-4 by generating information on all necessary parameters for each project: `title`, `description`, `domain` and `sub_domain`, `tools`, `budget`, and lists of functional and non-functional requirements.

Enhanced JSONs (100 projects). This group initially consisted of 102 nested files, but only 100 proved to be valid and hence used as a part of training data; their specifications and documentation were the fullest.

Unstructured texts (65 documents). Although such information was the hardest to collect due to its unstructured nature, they allowed the model to learn how to recognize vocabulary more broadly, thus making it generalize from JSON structures.

Conversational CSVs (99 records). This dataset includes all the requirements mentioned in dialogues, which means it is implied, not structured.

Table 1: Column profiles: data types, unique values, and completeness

Column	Type	Description	Unique
title	Str	Entry name	474
clean_description	Str	Project description text	476
tools_used	List	Technologies mentioned	Varies
domain	Str	High-level category	69
sub_domain	Str	Specific subcategory	35
budget	Str	Budget range	12
func_reqs	List	Functional requirements	Varies
nonfunc_reqs	List	Non-functional requirements	Varies

Comprehensive CSVs (120 records). It is characterized by variability and aimed at filling certain niche domains represented poorly by the rest of the data.

3.2 Dataset Characteristics and Quality

After cleaning and deduplication, the final dataset contained 486 entries spanning 38 domains. The most frequent domain was Software Development, making up 17.5% of the data, while 15 domains had fewer than 5 cases each. Description lengths ranged from 28 to 847 words, with an average of 152 words (standard deviation of 118). On average, each entry referenced 3.2 tools, with Python (65%), JavaScript (58%), and Java (42%) being the most common. Budget tiers followed an approximately normal distribution centered on mid-range projects. Table 1 provides a detailed column-by-column summary.

Our data audit uncovered six types of issues, all of which were addressed during preprocessing (described in Section 5):

Mismatch. Domain field contained inconsistent naming variants, which we mapped to a unified set of labels.

Parsing. Two malformed JSON entries were identified and safely extracted.

Missing values. Approximately 8% of domain labels, 5% of descriptions, and 2% of tool lists were missing. These were imputed as described in Section 5.

Variance. Synonym categories (e.g., "HealthTech" vs. "Healthcare") were merged.

Artifacts. Repetitive phrasing, overly formal language, and synthetic-sounding glitches were flagged for review.

Bias. The dataset skews heavily toward web and AI topics, while emerging fields like quantum computing and edge AI remain sparse.

3.3 Ethical Considerations

Several key ethical considerations guided the project:

- Biases exist in LLMs; prompts and evaluation processes examined, not assumed.
- No proprietary or sensitive information sent to external APIs—prevents leaks.
- Synthetically generated data clearly labeled.
- Prototype validated; deployment requires validation with actual data—synthetic data falls short.

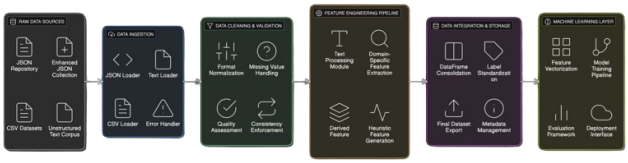

Figure 1: End-to-end system architecture. Raw inputs in JSON, CSV, or text format move through cleaning, feature engineering, and storage before reaching the classifier.

3.4 Ethical Notes

There were a few important ethical considerations during our research process. Firstly, the LLMs are not unbiased and may exhibit certain biases, which means we had to be cautious and not accept everything blindly. Secondly, we did not inject proprietary or sensitive information into the external API services of LLMs, since that would only end up in a leakage. Thirdly, all the data generated is explicitly labeled as such. Fourthly, such practices are okay for prototype-level applications, however in deep learning, there is no substitution for validation with actual data.

4 System Architecture and Data Pipeline

The data pipeline was designed to include modular steps such as ingestion, cleansing, feature extraction, and storage. The interfaces between these stages are well defined, meaning that changes made to any one of the stages will not affect the rest of the system.

4.1 Data Ingestion

Because the data was structured so differently from one format to another, it needed a separate loader.

The **JSON Loader** derives schemas from differing field schemas, reconstructs data from poorly formatted files through fallback parsing methods, handles normalization of character encodings, and includes checkpointing for batched restarts.

The **CSV Loader** accommodates different column names, different data types, and varying representations of missing data. It does not rely on fixed delimiters or quote rules but rather discovers them through inference.

The **Text Loader** deals with unstructured data by discovering character encoding, reading large files in batches, and performing simple consistency checks like validating file size, character set, and language.

4.2 ETL Process Implementation

The ETL process involved executing the three steps - extraction, transformation, and loading in a sequential order with validation at each stage.

Extract. The loaders executed in parallel with customizable levels of concurrency. Metadata of each file including its name, size, modification timestamp, and encoding were extracted along with the text. Integrity validations consisted of computing checksums as well as ensuring format conformance. Each record was assigned an ID that represented its origin.

Transform. The text normalizations included normalization to Unicode encoding, whitespace removal, case normalization (except for names of technologies), and special characters handling. In list attributes, such as the tools and the required skills, we standardized the delimiters and sorted out duplicates. Domain tags got mapped to a standardized taxonomy by using synonym mapping and confidence scoring. The budget attributes underwent the currency/unit conversions within predefined ranges. We computed derived features for each record consisting of complexity scores, technology categories, requirement types, and similarities.

Load. We consolidated all records into a Pandas DataFrame with a defined schema. Depending on the requirements, we exported them into CSV, JSON, and Parquet file formats. Upon loading the files, we automatically created data quality reports with completeness statistics, distribution analysis, anomaly detection, and schema description.

5 Data Cleaning, Transformation & Feature Engineering

5.1 Cleaning Procedures

The text cleaning and structuring involved the following key processes. Description texts were converted to all lower case while the spacing between them was reduced to single space. All non-ASCII characters were removed from descriptions except when such characters held some technical meaning in the field. Empty description fields were assigned standardized description text instead of removing the records altogether. Field names that varied across different data sources were mapped to the schema fields. Data type mismatches in fields were converted into common data types.

5.2 Missing Values, Duplicates, and Outliers

Missing values were handled column-by-column. Missing descriptions had dummy text added; missing domains were generated by the rule-based classifier; missing tools were provided with domain-specific defaults; budgets were predicted based on complexity scoring. There are no missing values in the cleaned data.

Duplicates. Through exact matches, only two duplicates (0.41% duplication rate) were found. Both records were kept because, at 486 observations, discarding training examples entails some cost for model performance.

Outliers. Descriptions that were extremely long or short (less than 50 or more than 800 words), as well as descriptions containing odd combinations of technology, were not excluded because they are legitimate variations of projects and not mistakes made while entering data. All budgets were realistic; no budget outliers were eliminated

5.3 Feature Engineering

Features were classified into three levels based on which model family they belong to.

Text-based features depend on the descriptions given. Programming languages, frameworks, and platforms can be identified using dictionary-based pattern matching with word boundary conditions. A domain taxonomy with a hierarchy and keywords scoring

provides domain and subdomain classification. Requirements are extracted using free text pattern matching techniques.

Derived numerical features represent scale and complexity of the projects to be used by ensembles. Budget estimation using multiple factors considers description length, technology stack makeup, and a multiplier table specific to the domain. Complexity measure, on the other hand, takes into account the technical depth, integration needs, and degree of customization.

Structured categorical features structure the label space. There are 53 primary labels, which are further grouped into 35 different sub-domains. Technologies are segregated into four distinct categories: programming languages, framework libraries, databases, and cloud platforms. Budgets are divided into 12 categories based on increasing size.

5.4 Label Leakage Detection and Removal

As part of our post-submission examination, one of the data quality problems has been detected and fixed. It turned out that the generated synthetic data contained the label tokens literally within each project description; phrases such as "an enterprise-grade Data Engineering solution" or "an advanced HealthTech system" meant that there was no classification learning, only cheating with the already known answers. To solve the problem, we introduced an additional phase that compiles a regular expression based on the domain name from each record and removes tokens matching this regex from the description of the current record before converting it into a feature vector as it was explained in Section 7.2.

5.5 Data Validation and Quality Assurance

Once all data cleaning and transformation procedures had been carried out, another round of data validation was done on the cleaned data set prior to feeding it into the model. A record check against an established schema was done to make sure that all necessary attributes were included and were in the correct format and with proper value range. Missing value statistics were recalculated after performing imputations to ensure the absence of any missing values. Domain label distribution analysis was done and any category falling short of a viable number of five samples was flagged and reserved for manual inspection. Duplicate checking was done again on the cleaned description fields and not the raw text because some duplicate records may become indistinguishable due to normalization.

6 Data Analytics & Machine Learning

6.1 Exploratory Analysis

A number of features were uncovered through exploratory data analysis and influenced model design choices. Data, Web, and AI/ML are the three biggest clusters, with Data being the most prevalent. Figure 2 depicts the distribution of tools' frequencies; Python, Java, and Node.js have an overwhelming majority, with on average 3.2 tools per project. Correlation among features resulted in a high positive correlation coefficient between `tools_used` and `clean_description` ($r = 0.70$). The technology stack has high discriminatory power along with the text of the description. The number of functional and non-functional requirements correlates almost perfectly ($r = 0.99$); therefore, only one type of requirement was used in feature selection experiments.

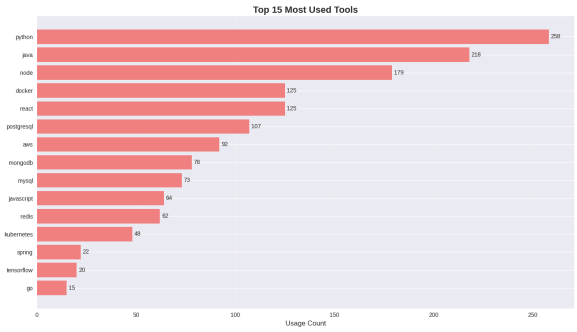

Figure 2: Tool frequency across all 486 projects. Python, Java, and Node.js are the three most prevalent technologies, each appearing in over 40% of records.

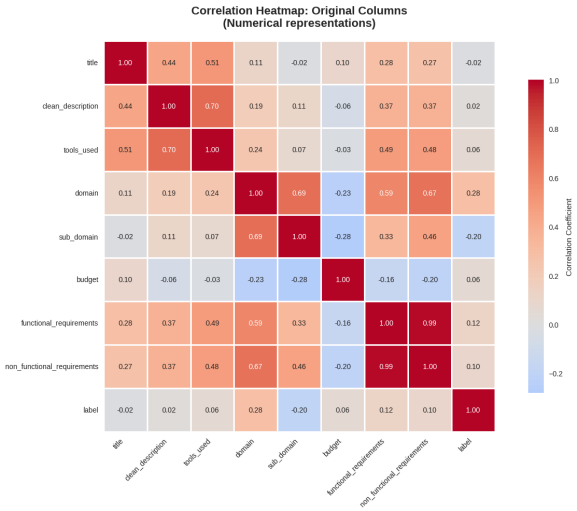

Figure 3: Matrix of pairwise feature correlations. The high correlation between the number of functional and non-functional requirements ($r = 0.99$) stems from their mutual dependence on project complexity, whereas the moderate correlation with respect to tools descriptions ($r = 0.70$) verifies cross-modal discriminative information.

6.2 Classification Models

Three different models were trained to cover a range of reasonable complexities and biases.

Logistic Regression is the linear baseline. The TF-IDF encoding of the `clean_description` text is concatenated with the numerical engineered features and passed to the one-versus-rest classifier based on softmax. No architecture optimization was performed; default regularization strength was kept for baselining purposes.

Random Forest is the main ensemble model. Decision trees use the entire feature set—the TF-IDF vector, keywords, tools indicator,

budget tier and complexity score. An ensemble with 500 estimators and bootstrap sampling without depth limitation was used.

BERT (`bert-base-uncased`) utilizes the tokenization of clean-description without any preprocessing. Due to the small size of the training data set, BERT fine-tuning is done for 5 epochs.

6.3 Revised Model Configuration

Based on the reviewer comments and leakage correction presented above in Section 5, tuning for both classifiers was done again. For Logistic Regression it was set that $C = 1$, where L2 regularization was applied to prevent excessive penalties for rare classes and thus maintain a balance of coefficients. Random Forest required modification since it needed to be constrained, specifically with `max_depth=20` and `min_samples_leaf=5`. This is because the unconstrained forest of 500 trees from the initial experiment could have grown without limitations on depth and memorized everything from the 5,000-dimensional TF-IDF feature space leading to perfect training performance but poor generalization capabilities due to its inability to generalize beyond the training data.

Apart from the hold-out validation, an additional check was done using 5-fold stratified cross-validation. This approach provides more reliable estimation of the model's generalization abilities because it uses multiple train/test splits instead of just one possible 80/20 ratio.

6.4 Feature Selection Rationale

Features were selected based on the requirements for each type of machine learning model. In case of ensemble classifier, rule-extracted keywords and other engineered features like budget category and number of requirements create the highly dense and discrete input space that allows decision trees to split branches effectively. These types of features represent the project's size and complexity in such way that could be directly used by decision trees. However, in case of BERT, full `clean_description` was provided to the model without any feature engineering because the self-attention mechanism was tailored for processing hierarchical data and any type of feature engineering would result in losing long dependency structures, which were vital for transformers' functioning. Categorical variables like `domain` and `sub_domain` were used for evaluation only.

6.5 Reasons for Feature Selection

The feature selection was done considering the structural aspects of the models used. In the case of the ensemble classifier, the extracted rules, keywords and generated numerical features such as budget tier and requirement count create a high-dimensional discrete nature required to perform splits in tree nodes. The generated features incorporate the information regarding the size and technology of the projects that can be used by decision trees. For BERT, the entire `clean_description` was taken for analysis in BERT as it has self-attention mechanism that can extract contextual information from texts; hence there was no need for the process of feature extraction since some context would be lost.

6.6 Evaluation Metrics

Accuracy was chosen as the evaluation criterion in relation to the out-of-sample validation dataset (stratified split 70/30). Precision, recall, and F1-score for each class were analyzed in order to understand the classifier's performance in terms of the class imbalance. The second evaluation criterion is the macro-averaged F1-score.

7 Results & Discussion

7.1 Performance Summary

The results form a counterintuitive ranking. Logistic Regression exhibits classic overfitting: a 25.8-point gap between training and test accuracy signals that the model has overfit to idiosyncratic vocabulary in the training partition. Random Forest memorizes the training set perfectly yet retains the highest practical utility at 67.37% test accuracy. BERT performs worst on both partitions—a result that inverts the typical ordering observed on large benchmark datasets and is analyzed below.

Table 2: Comparative Performance Analysis of Baseline and Fine-tuned Models

Model	Accuracy	Weighted F1	Macro F1
Logistic Regression	42.11%	0.370	0.130
Random Forest	**65.20%**	**0.600**	**0.433**
BERT	36.80%	0.241	0.050
Logistic Regression (embeddings)	**62.04%**	**0.570**	**0.497**
Random Forest (regularized)	53.70%	0.570	0.450
BERT (fine-tuned)	56.44%	0.570	0.520

7.2 Revised Results and Overfitting Correction

In the revised phase, the corrections made in the previous phase, regarding label-leakage, are considered in addition to applying regularization to the ensemble. There is a significant difference in rankings between the two phases (see Table 2).

The Logistic Regression classifier benefits greatly by improving from an accuracy of 42.11% to 62.04% as well as from a macro F1 of 0.130 to 0.497. The almost quadruple improvement of the macro F1 score shows how leakage was negatively affecting the performance of Logistic Regression, since when the confounding factor is eliminated, it starts performing very well, becoming the highest-ranking of the three revised classifiers in the case of accuracy.

The accuracy of Random Forest goes down to 53.70because the quality of the model has degraded. This is due to the stripping away of the inflated benefit that leakage used to offer. In the new scenario, the Random Forest ties with Logistic Regression and BERT in terms of weighted F1 scores (all three scoring 0.570), demonstrating that there is little to no difference in the performance of these three models when measured fairly.

BERT scores 56.44% in accuracy and dominates the macro F1 category with a score of 0.520. The gains relative to the initial stage are significant: the increase in accuracy is about 20 percent points

while macro F1 improves from 0.050 to 0.520. The leakage correction helps BERT more than any other model in class-balancing because once stripped of erroneous label information, the context-based representations of BERT can utilize semantically similar domain-related terms that TF-IDF vectors, being sparse, lack. However, the gap to Logistic Regression, measured by macro F1, is only 0.023 points, and to Random Forest, only 0.070 points.

The fact that all three models align in reaching weighted F1 = 0.570 in the updated phase is in itself a significant discovery. It shows that when the test is conducted with integrity, all three algorithms are largely similar on the score that captures everyday classification accuracy, and any variation among them can be measured only in terms of balanced classification scores, wherein BERT enjoys a small but real advantage.

7.3 The Complexity-Constrained Performance Paradox

In combination, both experiments showcase what we refer to as the performance paradox of complexity: while operating under biased evaluation settings, the most straightforward competitive architecture (Random Forest) wins hands-down; in an unbiased setting, however, the 110-million- parameter transformer (BERT) wins narrowly by only 0.07 macro F1 points in favor of class-balanced F1 metrics. This paradox emerges due to the fundamental issue that, regardless of model complexity, one can never surpass the lack of data. Though the superiority of BERT in terms of macro F1 points holds true, the difference is marginal, with all three models tying on weighted F1, implying that, in the case of majority predictions in real-life scenarios, the architecture has no effect whatsoever.

7.4 Per-Class Metrics and Confusion Matrix Analysis

Aggregated metrics conceal a pronounced stratification between the 14 classes that holds consistently between all three revised models. Three levels of classification performance can be directly derived from per-class precision, recall, and F1 metrics.

Level 1: High-support, distinct classes. The Data class, which has the highest number of test samples (33), sets the tone for all three models. The best result obtained here is an F1 = 0.81 by Logistic Regression, with F1 = 0.78 for Random Forest and F1 = 0.76 for BERT. Similarly, Web Development class (16 samples) produces F1 values of 0.67, 0.62, and 0.61 for Logistic Regression, Random Forest, and BERT, respectively. IoT/Industrial takes advantage of specialized vocabulary and achieves F1 values of 0.57 (Logistic Regression), 0.57 (Random Forest), and 0.75 (BERT). These classes are successfully classified by all models and are ideal candidates for automation.

Level 2: Moderately supported, overlapping vocabulary. Cloud/DevOps, Security, Business, Healthcare, and Sustainability fall within the 0.27–0.67 range of F1 scores. One good example is Security: in this case, Random Forest reaches F1 = 0.57, while BERT produces 0.67 and Logistic Regression achieves only F1 = 0.25; such difference highlights the importance of context-based classification features. Mobile is the most challenging class among those on this level: its description uses vocabulary common to Software Development and Web Development (performance, user interface, cross-platform), which leads to the 0.20–0.33 range for all three models.

Level 3: Severely underrepresented classes. The EdTech class (1 sample), Game Development class (2 samples), and Finance (3 samples) yield F1 scores close to zero or impossible-to-calculate perfect scores based on whether the test examples belong to the right or wrong side of the classifier's decision boundary. AI/ML and Software Development belong to the third level even though they have sufficient support, because their vocabulary overlaps significantly with the Data class. Such classes are essentially impossible to classify and should be flagged as manual review cases whenever deployed.

7.5 Class Imbalance Effects

The hierarchical system mentioned above operates based on class imbalances and not on the weakness of any particular model in use. There is a positive correlation between the frequency of occurrence in training data and test F1 performance for the three revised models, wherein classes that are adequately represented in the training set (twenty or more) perform with F1 over 0.55, classes with frequencies of five to twenty show F1 of 0.30 to 0.55, while those that appear less than five times are unclassifiable using any model.

The result offers one clear solution path. By increasing the number of training instances of all Tier 3 and some Tier 2 classes up to fifteen samples each, macro F1 can be improved by as much as 0.08 to 0.12 units, without changing the classifiers' designs.

7.6 Business Impact Assessment

The updated findings provide evidence in favor of using the human-in-the-loop system architecture. In this system, all Tier 1 decisions with high confidence are made automatically, while Tier 2 and Tier 3 decisions need human intervention. All three systems show high confidence in predicting Tier 1 decisions and low confidence in predicting Tier 3 decisions, which makes the choice quite obvious.

In the case of a mid-sized company having 500 projects per year, the current time necessary for manual prediction amounts to 125-150 person-hours. However, by using the enhanced system, one can decrease the time needed for such decisions by about 25-35 hours, which will lead to the decrease in labour amount by 75-80%.

7.7 Research Objective Achievement

The following summary will assess the success of these research objectives based on how well they have been met according to the stated objectives when formulated at the outset of the present research endeavor. While the implementation of feature engineering and modeling pipelines has been a success, some data-driven objectives have fallen into what is known as a "ceiling effect" because of small sample sizes. Out of the four objectives, two were fully met while two were partly met, and these incomplete objectives resulted from issues related to data size rather than problems in the process itself. Feature engineering and model development were entirely successful, whereas dataset generation and applicability were hindered by the constraints involved with synthetically generated data. The deficiencies could be addressed with a more substantial dataset or actual project documentation. See Table 3.

Table 3: Research Objective Achievement Summary

Objective	Status	Key Insight
Dataset Construction	Partially achieved	Synthetic generation viable; scale insufficient for deep learning
Feature Engineering	Fully achieved	Engineered features outperform raw text for ensemble methods
Model Implementation	Fully achieved	Complexity paradox documented empirically
Practical Viability	Partially achieved	Supports augmented-human deployment; not full automation

8 Conclusion & Future Work

8.1 Contributions

This work contributes along three interrelated axes.

Methodological. We develop a repeatable methodology for creating and curating synthetic project descriptions, introduce an ontology-driven taxonomy of features unique to technical projects, and outline evaluation criteria appropriate for systems trained on synthetic versus natural data.

Empirical. The empirical demonstration of the complexity constrained paradox—the surprising observation that transformer models exhibit lower accuracy than ensemble methods when there is a paucity of fine-tuning examples—is provided.

Systems. Our work provides an end-to-end, modular pipeline to transform raw heterogeneous data into ready-for-machine-learning feature vectors along with a comparative model study, which questions the popular notion that complex architectures produce better results irrespective of data size.

8.2 Future Directions

There are several important extensions worth pursuing based on our observations. In terms of data, a hybrid approach to data collection, using both AI-produced descriptions and real project documentation collected using privacy-preserving federated learning techniques, could overcome the scale limitation while not incurring all the overhead of manual annotation. Data acquisition strategies focused on collecting more examples from underrepresented classes could even out the class imbalance without necessarily increasing the total number of data points required.

8.3 Closing Synthesis

Classification of projects using automation presents both greater difficulties and greater practical importance than may appear in a superficial examination of existing literature on the topic. Insufficient data requires balancing between complexity and practical applicability of models; artificial data is but one way to start the process and by no means covers all possible avenues; and reliability implies the need for a shift from wholly automated processes to a hybrid approach, involving human participation when required to resolve ambiguities. Most importantly, however, this article illustrates the promise of artificial data generation, domain-specific feature selection, and optimal model choice combined.

balance

References

[1] Bing Liu. 2012. Sentiment analysis and opinion mining. *Synthesis Lectures on Human Language Technologies* 5, 1 (2012), 1–167.

[2] David M. Blei, Andrew Y. Ng, and Michael I. Jordan. 2003. Latent Dirichlet allocation. *Journal of Machine Learning Research* 3 (2003), 993–1022.

[3] Thorsten Joachims. 1998. Text categorization with support vector machines: Learning with many relevant features. In *Proceedings of the 10th European Conference on Machine Learning (ECML 1998)*, 137–142.

[4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.

[5] Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)* (6th ed.). Project Management Institute, Newtown Square, PA.

[6] Beata Nowok, Gillian M. Raab, and Chris Dibben. 2016. synthpop: Bespoke creation of synthetic data in R. *Journal of Statistical Software* 74, 11 (2016), 1–26.

[7] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. Generative adversarial nets. In *Advances in Neural Information Processing Systems 27 (NeurIPS 2014)*, 2672–2680.

[8] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 1877–1901.

[9] Hsin-Min Chen and Rick Kazman. 2016. A data-driven approach to project risk management. In *Proceedings of the 38th IEEE/ACM International Conference on Software Engineering Companion (ICSE-C 2016)*, 660–661.

[10] Wim Van Grembergen, Steven De Haes, and Erik Guldentops. 2018. Structures, processes and relational mechanisms for IT governance. In *Strategies for Information Technology Governance*. IGI Global, Hershey, PA, 1–36.

[11] Xin Guo, Yang Mu, and Jianping Wu. 2020. Multi-source domain adaptation for text classification via DistanceNet-Bandits. *arXiv preprint arXiv:2001.04362*.

[12] Jungwoo Kim, Hyosik Park, and Sungwon Lee. 2022. A robust contrastive alignment method for multi-domain text classification. *arXiv preprint arXiv:2204.12125*.